

wxsum: A command for processing temperature and precipitation data

Jeffrey D. Michler	Anna Josephson	Oscar Barriga-Cabanillas
University of Arizona	University of Arizona	World Bank
Tucson, AZ	Tucson, AZ	Washington, D.C.
jdmichler@arizona.edu	jdmichler@arizona.edu	obarriga@worldbank.org
Aleksandr Michuda	Jeffrey C. Oliver	
Swarthmore College	University of Arizona	
Swarthmore, PA	Tucson, AZ	
amichud1@swarthmore.edu	jcoliver@arizona.edu	

Abstract. Weather data are now a standard input in socioeconomic analysis, yet computing seasonal weather metrics from daily data remains error-prone and time-consuming. In this article, we introduce `wxsum`, a community-contributed command that reduces this cost. The command takes daily precipitation or temperature data and calculates 23 precipitation and 14 temperature measures, ranging from summary statistics (mean, standard deviation) to shock measures (z-scores, dry spells, killing degree days). By standardizing these calculations, `wxsum` lowers the probability of coding errors and allows researchers to move quickly from raw weather data to analysis-ready variables.

Keywords: st0001, `wxsum`, Earth observation, weather data, data management, summary statistics

1 Introduction

The expanding availability and decreasing cost of use have made weather data, particularly remotely sensed and/or gridded data, a common input into socioeconomic analysis (Dell et al. 2014; Donaldson and Storeygard 2016). Economists use these ready-made Earth observation products to predict a range of economic outcomes, including agricultural and industrial output, labor productivity, energy demand, health, conflict, human capital, and economic growth. Yet, there are many ways to quantify weather and very little theoretical or systematic guidance on which weather metric is best suited to a given research question and geographic context. The result is that seemingly similar measures of “rainfall,” “temperature,” or “shock” can be based on very different summaries of the underlying weather data (Michler et al. 2021). The variety of metrics used and the size of weather data frames can create data-handling and cleaning problems, increasing the probability of coding errors when calculating any given metric.

In this article, we introduce the community-contributed command `wxsum`, which processes daily precipitation and temperature data and produces daily, monthly, seasonal, or annual summary statistics and shock measures. The goal of the command is to enable easy, error-free calculation of the most common weather metrics used in the

socioeconomic literature. The **wxsum** command takes in daily precipitation or temperature data, typically in the form of gridded Earth observation data, and calculates a number of summary statistics for both, such as the mean, median, variance, and skew, as well as phenomenon-specific statistics, such as total seasonal rainfall, maximum seasonal temperature, and growing degree days (GDD), and several measures of shocks, including deviations from long-run averages, z-scores, dry spells, and killing degree days (KDD).

The **wxsum** command allows the user to customize season dates, the rainfall threshold defining a rainy day, GDD bounds, KDD thresholds, daily-temperature bins, the length of the long-run reference window, and the final output shape (long or wide). This flexibility is important because the relevant exposure period, temperature sensitivity, and baseline climate vary across crops, agroecological zones, and research questions. For example, precipitation may matter most during planting and flowering stages, while heat exposure becomes critical during grain filling. The command accommodates these differences without requiring the user to reshape or pre-process the data beyond the standard wide format.

The choice of metric should follow from the research question and the mechanism under study. For applications focused on crop yields or agricultural output, growing degree days and killing degree days capture the accumulated heat exposure relevant to plant development and damage. GDD bins (Deschênes and Greenstone 2007) or temperature bins (Schlenker and Roberts 2009) provide a non-parametric approach to estimating the relationship between temperature and output. Total seasonal rainfall and rainy-day counts are natural measures when precipitation levels directly drive outcomes, as in studies of agricultural productivity or water availability. Deviations from long-run averages and z-scores are suited to identifying weather shocks, seasons that are unusual relative to local climate, and are common in studies of consumption, poverty, and conflict. Early, intra-seasonal, or late dry spells capture the timing and persistence of rainfall deficits, which may matter more than total rainfall for rain-fed agriculture. These are not exclusive categories; many applications combine several metrics to capture different dimensions of weather exposure. Additionally, we have developed an implementation for R (**wxsumR**), extending the same core functionality to users who integrate weather processing into R-based data pipelines.

While the preceding guidance reflects common practice in the literature, the command itself is agnostic about which of the many weather metrics a researcher uses. In creating the **wxsum** command, we are not endorsing an atheoretical or ad hoc approach to incorporating weather measures into socioeconomic analysis. A growing body of literature raises concerns about the use (and misuse) of weather data, particularly when weather is used to identify exogenous variation (Benami et al. 2026). These concerns include how gridded weather data is matched with socioeconomic data (Michler et al. 2022), measurement error in Earth observation products (Proctor et al. 2023; Josephson et al. 2026), and the validity of the assumption that weather is truly exogenous (Sarsons 2015; Mellon 2025). While incorporating weather data into one’s analysis is now relatively low cost, a naïve or unexamined use of that data will continue to carry a high cost in terms of the accuracy of one’s analysis and results.

2 Data considerations

The `wxsum` command is designed for a common structure of remotely sensed or gridded Earth observation products after they have been matched to locations in an analysis dataset. Each observation (row) is a location, plot, household, firm, or other spatial unit, and each variable (column) is a daily measurement for that unit. The command begins after the spatial matching step. Before running it, the user should decide which weather product, temporal aggregation, and spatial linking rule are appropriate for the research design. These choices are substantive, not merely computational, as estimates can be sensitive to the source of weather data and to the way gridded observations are linked to socioeconomic observations (Auffhammer et al. 2013).

In a typical workflow, the user extracts daily weather values from a gridded product at the GPS coordinates of each observation (using GIS software, R, or Python) and imports the resulting wide-format dataset into Stata. Users should consult Michler et al. (2022) and Michler et al. (2026) for guidance on spatial linking choices

The command requires Stata 15 or later. A dataset with 40 years of daily observations implies roughly 14,600 variables per observation. Stata/IC limits datasets to 2,048 variables, which restricts the command to approximately five years of daily data. Stata/SE supports up to 32,767 variables, and Stata/MP supports up to 120,000. Users working with long time series in Stata/IC may need to split the data into shorter panels, run `wxsum` on each, and append the results.¹

2.1 Shape and naming convention

Input data must be in wide form. Daily weather values are stored in separate numeric variables whose names share a common prefix (stub) and end with an eight-digit date in `yyyymmdd` format. For example, if the precipitation variables are named with the prefix `rf_`, then precipitation on May 15, 1979, should appear in a variable named `rf_19790515`. A similar approach applies for temperature variable names after the prefix. Variables not beginning with the specified stub are ignored unless they are retained by using the `keep()` option.

The wide format, while memory-intensive, is a natural representation of the gridded Earth observation products common in the climate science literature. The `wxsum` command is agnostic to the source of the data. It can process data from any product that provides daily or subdaily readings on a regular grid. Widely used products of this type include CHIRPS (Funk et al. 2015), CPC (Chen et al. 2008), ARC2 (Novella and Thiaw 2013), TAMSAT (Tarnavsky et al. 2014), ERA5 (Hersbach et al. 2018), and MERRA-2 (Bosilovich et al. 2016). It can also process data from ground-based weather station networks.

The required naming convention for the input data makes the calendar explicit. The

1. Splitting the data affects the rolling deviations and z-scores, as the command requires `lr_years()` of prior seasons to be present in the same dataset. Users who split their data should ensure sufficient overlap to avoid truncating the long-run reference period

command parses the date suffix of each weather variable, identifies the days that belong to each user-defined season, and computes statistics separately for each location and each season. As all computations are based on the dates in variable names, users should check that the date suffixes are complete, unique, and measured consistently on the intended calendar and time zone. The command can accommodate seasons that cross calendar years. For example, when a season spans two calendar years, it is assigned to the year in which it begins — a season running from November 2001 to February 2002 is labeled 2001 in the command’s output.

2.2 Seasons, units, and missing values

A season is defined by an initial month and final month and, optionally, by an initial day and final day. This allows for the relevant period for rainfall or heat exposure to differ across crops, agroecological zones, and outcomes of interest. Users can specify the “season” as an entire growing season or a specific section of it. The command does not impose a biological or econometric interpretation on the chosen season. Rather, the user must define the exposure window before running the command and should report that choice in subsequent empirical work.

The command is similarly agnostic about units. Rainfall can be measured in millimeters, centimeters, or inches, and temperature can be measured in degrees Celsius, Fahrenheit, or Kelvin. It is necessary, however, that all daily variables are measured consistently with the same unit used throughout. Unit choices matter for thresholds. For example, the lower and upper bounds for GDD and the base for KDD must be expressed in the same units as the daily temperature data. Similarly, the rainy-day threshold must be expressed in the same units as the daily precipitation data.

Many weather products contain missing values due to sensor problems, limited spatial coverage, cloud contamination, or limitations in station interpolation (Dinku 2020). Before using **wxsum**, users should examine missingness in the daily variables and decide whether the weather product and temporal coverage are adequate for the application. For means, medians, standard deviations, maxima, rainy-day counts, rain-day shares, daily-temperature bin counts, and dry spells, missing daily values are not treated as observed weather. For seasonal and monthly precipitation totals, **wxsum** uses Stata’s **rowtotal()** function, and so missing daily values contribute zero to those sums. This means seasons with missing days will understate true missingness in total rainfall. If an empirical design requires a balanced panel of weather exposures or a minimum number of observed days within a season, that restriction should be imposed before running **wxsum** and should be documented in the replication files.

3 The **wxsum** command

3.1 Syntax

The syntax is

```
wxsum stubname , type(rain|temp) ini_month(month) fin_month(month) [
    ini_day(day) fin_day(day) rain_threshold(num) gdd_lo(num)
    gdd_hi(num) kdd_base(num) gdd_bin(num) gdd_binlo(num)
    gdd_binhi(num) tmp_bin(num) tmp_binlo(num) tmp_binhi(num)
    shape(wide|long) lr_years(num) keep(varlist) save(filename) ]
```

Here *stubname* is the common prefix for the variables. It is not a varlist. If the data contain variables named `rf_19790101`, `rf_19790102`, and so on, the *stubname* supplied to the command is `rf_`. The `type()` option must be specified as either `rain` or `temp` and the initial and final months must be set.

3.2 Required variables

The minimal dataset contains an identifier for each location and a set of variables that contain daily measures of precipitation or temperature. The identifier is not an argument of `wxsum`; it can be retained with `keep()` if it should appear in the output dataset. The precipitation or temperature variables must be numeric and must use the naming convention described above. Table 1 summarizes these requirements.

Table 1: Data requirements for `wxsum`

Element	Requirement
Observation	One row per spatial or analytic unit.
Weather variables	One variable per day, with a common stub and a <code>yyyymmdd</code> date suffix.
Date information	Encoded in the daily variable names rather than stored in separate year, month, and day variables.
Data type	Numeric daily precipitation or temperature variables specified using <code>type(rain)</code> or <code>type(temp)</code> .
Other variables	Retain with <code>keep()</code> if identifiers, coordinates, or covariates should remain in the output.

3.3 Options

`type(rain|temp)` specifies the data type. Use `rain` for precipitation data or `temp` for temperature data. This option is required.

`ini_month(month)` specifies the initial month of the season. This option is required.

`fin_month(month)` specifies the final month of the season. This option is required. If the final month precedes the initial month in the calendar year, the command treats the season as crossing calendar years.

- ini_day(day)** specifies the first day of the initial month to be included in the season. The default is 01.
- fin_day(day)** specifies the last day of the final month to be included in the season. If not specified, the command dynamically defaults to the true last calendar day of the final month (handling leap years correctly).
- lr_years(num)** specifies the number of strictly preceding years used to compute rolling deviations and *z*-scores. The default is 10, and the maximum is 50.
- rain_threshold(num)** specifies the threshold used to define a rainy day. The default is 1. Valid only with **type(rain)**.
- gdd_lo(num)** specifies the lower bound for growing degree days. This option is required when **type(temp)** is specified.
- gdd_hi(num)** specifies the upper bound for growing degree days. This option is required when **type(temp)** is specified.
- kdd_base(num)** specifies the threshold above which killing degree days are calculated. This option is required when **type(temp)** is specified.
- gdd_bin(num)** specifies the width (in GDDs) of seasonal GDD bins. When specified, the command creates an integer categorical variable **gddcat_YYYY** for each season. The argument is a positive numeric bin width, expressed in the units of the generated GDD variable. Valid only with **type(temp)**.
- gdd_binlo(num)** specifies the optional lower endpoint for GDD bins. The default is 0 when **gdd_bin()** is specified. If any seasonal GDD total falls below this value, a bottom-coded bin is created. Requires **gdd_bin()**.
- gdd_binhi(num)** specifies the optional upper endpoint for GDD bins. Values at or above this endpoint are assigned to a top-coded bin. When omitted, the command determines an upper endpoint from the pooled empirical maximum of generated seasonal GDD values and the chosen bin width. Requires **gdd_bin()**. The range between **gdd_binhi()** and **gdd_binlo()** must be evenly divisible by **gdd_bin()**.
- tmp_bin(num)** specifies the total number of daily temperature bins to create per season. Must be a positive integer from 1 through 42. Requires **tmp_binlo()** and **tmp_binhi()**. Creates variables **tmpbin01_YYYY** through **tmpbinJJ_YYYY**. Valid only with **type(temp)**.
- tmp_binlo(num)** specifies the lower endpoint for the first daily temperature bin. Required when **tmp_bin()** is specified. No default.
- tmp_binhi(num)** specifies the upper endpoint for the last daily temperature bin. Required when **tmp_bin()** is specified. Must be greater than **tmp_binlo()**. No default.

shape(wide|long) specifies the final output shape. The default is **wide**, which produces one row per spatial unit with year-suffixed variable names. When **long** is specified, the output is stacked into one row per retained unit-year and a variable named **year** is created. Input data must still be wide. The **shape(long)** option is a final-output stacking operation; it does not change the internal computation strategy or reduce peak memory use.

keep(varlist) retains variables from the input dataset in the output dataset. This option is typically used for identifiers, administrative codes, coordinates, or other merge keys.

save(filename) saves the resulting dataset to *filename*. If **save()** is not specified, the generated variables remain in memory.

3.4 Output

wxsum is a data-management command rather than an estimation command. Its primary output is a dataset containing the retained variables requested in **keep()** and the generated seasonal weather variables.

In the default **shape(wide)** output, the command generates one set of variables for each season in the data, so the number of output variables increases with the time span covered by the daily weather data. Therefore, using a dataset spanning 30 years of daily rainfall will produce 30 instances of each of the 23 precipitation statistic. Generated variables are indexed by the season-start year using a **_YYYY** suffix. For example, if a season begins in November 2001 and ends in February 2002, the resulting seasonal statistics are associated with 2001, not 2002.

When **shape(long)** is specified, the output is stacked with one row per retained unit-year. A variable named **year** identifies the season-start year. Generated variables are renamed without their year suffixes; for example, **mean_1993** becomes **mean** with **year == 1993**. Variables specified in **keep()** are repeated across years. Users are strongly encouraged to include unit identifiers in **keep()** when using **shape(long)**, because otherwise the long output may not contain an explicit merge key. The **shape(long)** option does not change the requirement that input data be wide; it is a final-output stacking step only.

When **type(rain)** is specified, the command produces the precipitation statistics in Table 2. These variables combine simple summaries of the daily distribution, summaries of monthly totals, seasonal totals, rainy-day counts, and shock measures relative to the rolling long-run reference period.

When **type(temp)** is specified, the command produces the temperature statistics in Table 3. The growing degree day calculation uses **gdd_lo()** and **gdd_hi()** as lower and upper bounds, whereas killing degree days are calculated relative to **kdd_base()**. GDD categories, generated when **gdd_bin()** is specified, classify the seasonal GDD total into bins of fixed width. Daily temperature bins, generated when **tmp_bin()**

Table 2: Rainfall statistics generated by **wxsum**

Group	Generated statistics
Daily distribution	Mean, median, variance, standard deviation, and skewness of daily precipitation within the season.
Monthly totals	Mean monthly precipitation within the season, deviation from the long-run average of mean monthly precipitation, and its z -score.
Seasonal total	Total precipitation in the season, deviation from the long-run average of total precipitation, z -score of total precipitation.
Rainy days	Number of rainy days, number of days without rain, deviations from the corresponding long-run averages, and corresponding z -scores.
Rainy-day share	Share of observed days with rain, deviation from the long-run average share, corresponding z -score.
Dry spells	Length of the leading dry spell, longest mid-season dry spell, and length of end of season dry spell.

is specified, count the number of observed daily temperature readings in each fixed absolute temperature interval.

The command calculates the non-reference variables whenever the selected daily variables and season are available. Rolling deviations and z -scores require sufficient prior history at the same location. Consequently, if the data begin in 2000 and the user requests `lr_years(10)`, the first eligible long-run deviations are not calculated until the season beginning in 2010. This is intentional: the current season is never included in the reference period against which it is compared.

4 Formulas and Calculations

This section defines the variables generated by **wxsum**. Let i index the observation in the dataset and let t denote the year in which a season begins. Let S_t be the set of daily dates included in the season beginning in year t , and let N_{it} be the number of non-missing daily observations for observation i in season t . For rainfall variables, let R_{id} denote daily precipitation on date d ; for temperature variables, let T_{id} denote daily temperature. The command creates variables with the year suffix `_YYYY` but in order to simplify notation in this section, we let `_t` stand for the year, so the formulas below describe, for example, `mean_t` instead of `mean_YYYY`.

Table 3: Temperature statistics generated by `wxsum`

Group	Generated statistics
Daily distribution	Mean, median, variance, standard deviation, skewness, maximum daily temperature within the season.
Growing degree days	GDD, deviation from the long-run average of GDD, z -score of GDD.
Killing degree days	KDD, deviation from the long-run average of KDD, z -score of KDD.
GDD categories	Integer category variable <code>gddcat_YYYY</code> identifying fixed-width bins of the seasonal GDD total, with Stata value labels defining the intervals.
Daily temperature bins	Count variables <code>tmpbin01_YYYY</code> through <code>tmpbinJJ_YYYY</code> recording the number of observed daily temperature readings falling in each fixed absolute temperature interval.

4.1 Statistics common to rainfall and temperature

For either rainfall or temperature data, write X_{id} for the relevant daily variable, either R_{id} or T_{id} . The daily sample mean is

$$\text{mean}_{.t_i} = \frac{1}{N_{it}} \sum_{\substack{d \in S_t \\ X_{id} \neq \text{missing}}} X_{id}. \quad (1)$$

The daily median, `median_{.t}`, is the median of the non-missing values $\{X_{id} : d \in S_t, X_{id} \neq \text{missing}\}$. The daily variance, `var_{.t}`, requires at least 2 non-missing observations and is the sample variance of those same non-missing values,

$$\text{var}_{.t_i} = \frac{1}{N_{it} - 1} \sum_{\substack{d \in S_t \\ X_{id} \neq \text{missing}}} (X_{id} - \text{mean}_{.t_i})^2. \quad (2)$$

The daily standard deviation, `sd_{.t}`, is the square root of the variance. The skewness variable is the adjusted Fisher-Pearson sample skewness,

$$\text{skew}_{.t_i} = \frac{N_{it}}{(N_{it} - 1)(N_{it} - 2)} \sum_{\substack{d \in S_t \\ X_{id} \neq \text{missing}}} \left(\frac{X_{id} - \text{mean}_{.t_i}}{\text{sd}_{.t_i}} \right)^3. \quad (3)$$

The third moment variable requires at least 3 non-missing observations. When $N_{it} < 3$ or the standard deviation is zero, the skewness variable is missing.

4.2 Rainfall calculations

For precipitation data, the seasonal total is

$$\text{total_}t_i = \sum_{d \in S_t} R_{id}, \quad (4)$$

where the implementation follows `rowtotal()` and therefore treats missing daily rainfall components as zero in the sum.

Monthly totals are computed first for each calendar month m represented in the selected season,

$$M_{imt} = \sum_{\substack{d \in S_t \\ \text{month}(d)=m}} R_{id}. \quad (5)$$

The variable `mean_mo_t` is the mean of the monthly totals M_{imt} across the k_{it} months represented in the season.

Let c denote the value supplied in `rain_threshold()`. Rainy days and days without rain are counted only among non-missing daily rainfall values:

$$\text{raindays_}t_i = \sum_{\substack{d \in S_t \\ R_{id} \neq \text{missing}}} \mathbf{1}\{R_{id} \geq c\}, \quad (6)$$

$$\text{norain_}t_i = \sum_{\substack{d \in S_t \\ R_{id} \neq \text{missing}}} \mathbf{1}\{R_{id} < c\}. \quad (7)$$

The share of observed days with rain is

$$\text{pct_raindays_}t_i = \frac{\text{raindays_}t_i}{N_{it}}. \quad (8)$$

The command calculate three different dry spells. The variable `dry_t` is the longest mid-season dry spell: the maximum run of consecutive non-missing days with $R_{id} < c$ that occurs strictly after the first observed rainy day and strictly before the last observed rainy day in the season. Leading dry days before the first rainy day and trailing dry days after the last rainy day are excluded from `dry_t`.

The variable `dry_start_t` is the number of consecutive non-missing dry days at the start of the season before the first rainy day or missing value. The variable `dry_end_t` is the number of consecutive non-missing dry days at the end of the season after the last rainy day or missing value, counted backward from the final day of the season. If all daily rainfall values in a location-season are missing, all three dry-spell variables are missing. Missing rainfall values break dry spells rather than extending them.

4.3 Temperature calculations

For temperature data, the maximum is:

$$\max.t_i = \max_{\substack{d \in \mathcal{S}_t \\ T_{id} \neq \text{missing}}} T_{id}. \quad (9)$$

Let a be the lower bound supplied in `gdd_lo()` and b be the upper bound supplied in `gdd_hi()`. Daily growing degree days are truncated below at zero and above at $b - a$, and seasonal GDD are accumulated as:

$$\text{gdd}.t_i = \sum_{\substack{d \in \mathcal{S}_t \\ T_{id} \neq \text{missing}}} \min \{ \max (T_{id} - a, 0), b - a \}. \quad (10)$$

If the user supplies a positive value k in `kdd_base()`, killing degree days are:

$$\text{kdd}.t_i = \sum_{\substack{d \in \mathcal{S}_t \\ T_{id} \neq \text{missing}}} \max (T_{id} - k, 0). \quad (11)$$

If `kdd_base()` is left at its default value of zero, `wxsum` does not generate the KDD variables.

GDD bins provide a nonlinear degree-day specification in the spirit of Deschênes and Greenstone (2007). This generated categorical variable identifies which fixed-width GDD interval the accumulated seasonal GDD total falls into. When `gdd_bin()` is specified, let GDD_{it} denote seasonal growing degree-days computed as above, and let $\tau_0 < \tau_1 < \dots < \tau_J$ denote fixed GDD cutpoints generated from `gdd_bin()`, `gdd_binlo()`, and either `gdd_binhi()` or the automatically determined upper endpoint. For regular categories:

$$\text{gddcat}_{it} = j \quad \text{if} \quad \tau_{j-1} \leq \text{GDD}_{it} < \tau_j. \quad (12)$$

If a bottom-coded bin is needed because some seasonal GDD totals fall below `gdd_binlo()`:

$$\text{gddcat}_{it} = 1 \quad \text{if} \quad \text{GDD}_{it} < \tau_0. \quad (13)$$

If a top-coded bin is requested via `gdd_binhi()`:

$$\text{gddcat}_{it} = J + 1 \quad \text{if} \quad \text{GDD}_{it} \geq \tau_J. \quad (14)$$

The variable `gddcat_YYYY` is an integer-valued categorical variable with value labels define the GDD intervals, so that `tabulate gddcat_YYYY` displays the interval

boundaries. Users can employ factor-variable notation, such as `i.gddcat_YYYY` in wide output or `i.gddcat` after `shape(long)`, to create regression indicators for each bin. The command is unit agnostic so `gdd_bin()`, `gdd_binlo()`, and `gdd_binhi()` must be expressed in the units of the generated GDD variable.

Daily temperature bins are another nonlinear measure that summarizes the seasonal distribution of observed daily temperatures across common absolute temperature intervals in the spirit of Schlenker and Roberts (2009). These variables count the number of nonmissing daily temperature readings falling in each interval and are created when `tmp_bin()` is specified.

For $J = \text{tmp_bin}()$ with $J \geq 3$, let $\ell = \text{tmp_binlo}()$, $h = \text{tmp_binhi}()$, and $w = (h - \ell)/(J - 2)$. The bin counts are:

$$\text{tmpbin}_{it}^{(j)} = \begin{cases} \sum_{\substack{d \in S_t \\ T_{id} \neq \text{missing}}} \mathbf{1}\{T_{id} < \ell\}, & j = 1, \\ \sum_{\substack{d \in S_t \\ T_{id} \neq \text{missing}}} \mathbf{1}\{\ell + (j - 2)w \leq T_{id} < \ell + (j - 1)w\}, & 2 \leq j \leq J - 1, \\ \sum_{\substack{d \in S_t \\ T_{id} \neq \text{missing}}} \mathbf{1}\{T_{id} \geq h\}, & j = J. \end{cases} \quad (15)$$

Interior intervals are lower-closed and upper-open. The lower tail is strictly below ℓ ; the upper tail is at or above h . The generated variable names are zero-padded, for example `tmpbin01_1993`.

Two special cases apply for small J . When `tmp_bin(1)` is specified, a single variable `tmpbin01_YYYY` counts all nonmissing daily temperature readings in the season. When `tmp_bin(2)` is specified, the command splits at the midpoint $m = (\ell + h)/2$: `tmpbin01_YYYY` counts days with $T < m$ and `tmpbin02_YYYY` counts days with $T \geq m$. Missing daily temperatures are not counted in any bin. If all daily temperatures for a location-season are missing, all bin variables for that location-season are set to missing. Otherwise, the sum of all bin counts equals the number of nonmissing daily temperature readings in the season.

4.4 Long-run reference periods

Several variables generated by `wxsum` describe weather shocks as deviations from a location-specific long-run reference period. Let w_{it} be one of the seasonal weather statistics for which a long-run benchmark is generated: `total_t`, `raindays_t`, `norain_t`, or `pct_raindays_t` for rainfall data; `gdd_t` and `kdd_t` for temperature data. Let L be the number of prior years requested in `lr_years()`. The rolling long-run mean and standard deviation used for deviations and z scores are based only on strictly preceding seasons:

$$\bar{w}_{it}^L = L^{-1} \sum_{l=1}^L w_{i,t-l} \quad (16)$$

$$s_{it}^L = \left\{ (L-1)^{-1} \sum_{l=1}^L (w_{i,t-l} - \bar{w}_{it}^L)^2 \right\}^{1/2}. \quad (17)$$

The deviation and corresponding z score are

$$\text{dev_w_t}_i = w_{it} - \bar{w}_{it}^L, \quad (18)$$

$$\text{z_w_t}_i = \frac{w_{it} - \bar{w}_{it}^L}{s_{it}^L}. \quad (19)$$

The use of strictly prior years avoids including the current realization in its own benchmark. If there are too few preceding season variables to satisfy the requested value of `lr_years()`, the long-run deviations and z scores are not generated for those initial seasons.

5 Examples

The package includes small rainfall and temperature datasets that can be used to check installation and illustrate the syntax. These files are also useful as templates for checking the required wide layout and variable-naming conventions.

5.1 Rainfall summaries

Suppose that the daily rainfall variables in `rain.dta` are named with the prefix `rf_`. The following commands calculate seasonal rainfall statistics for a May 15 through October 15 growing season and save the resulting dataset:

```
. use rain.dta, clear
. wxsum rf_, type(rain) ini_month(05) fin_month(10) ini_day(15) ///
>   fin_day(15) rain_threshold(1) keep(hhid) ///
>   save(rainfall_stats.dta)
```

The output dataset contains one observation for each input location. For each season observed in the daily rainfall variables, `wxsum` adds the rainfall statistics listed in Table 2. If the user also needs location identifiers or other merge variables, these should be supplied in `keep()`. The resulting file can then be merged with a socioeconomic dataset using the retained identifier variables and the season-specific weather variables generated.

5.2 Temperature summaries

Suppose that the daily temperature variables in `temp.dta` are named with the prefix `tmp_`. The following commands calculate seasonal temperature statistics for a November through February growing season. As the final month precedes the initial month, the command treats the exposure window as spanning calendar years and assigns the season to the year in which it begins:

```
. use temp.dta, clear
. wxsum tmp_, type(temp) ini_month(11) fin_month(02) gdd_lo(8) ///
>      gdd_hi(32) kdd_base(32) keep(hhid)
```

This command generates the temperature summaries listed in Table 3. The values `gdd_lo(8)`, `gdd_hi(32)`, and `kdd_base(32)` are illustrative; they should be chosen to match the units of the underlying temperature data and the biological or behavioral process being studied.

5.3 GDD and temperature bins

To create Deschênes–Greenstone-style GDD bins, the user specifies `gdd_bin()` with a bin width in the units of the generated GDD variable:

```
. use temp.dta, clear
. wxsum tmp_, type(temp) ini_month(04) ini_day(01) fin_month(09) ///
>      fin_day(15) gdd_lo(18) gdd_hi(30) kdd_base(32) gdd_bin(100) ///
>      gdd_binlo(50) gdd_binhi(950) keep(hhid)
```

Here `gdd_bin(100)` means the bin width will be 100 degree-days in the units of the generated GDD variable. In this example the user set `gdd_binhi()` and `gdd_binlo()` though these are not required. In the absence of these options, the default lower endpoint is 0 while the command determines the upper endpoint from the pooled empirical maximum of GDD. This example assumes Celsius temperatures and Celsius degree-days. The command does not infer or convert units.

To create Schlenker–Roberts-style temperature bins, the user specifies `tmp_bin()` along with `tmp_binlo()` and `tmp_binhi()`:

```
. use temp.dta, clear
. wxsum tmp_, type(temp) ini_month(04) ini_day(01) fin_month(09) ///
>      fin_day(15) gdd_lo(18) gdd_hi(30) kdd_base(32) tmp_bin(15) ///
>      tmp_binlo(0) tmp_binhi(39) keep(hhid)
```

Here `tmp_bin(15)`, `tmp_binlo(0)`, and `tmp_binhi(39)` create 15 bin-count variables per season: a lower tail counting days below 0, 13 equal-width interior intervals over $[0, 39)$, and an upper tail counting days at or above 39. This is a Celsius-style example; the command does not infer or convert units.

5.4 Changing the long-run benchmark

The default long-run reference period is 10 prior years. Researchers working with longer weather histories may prefer a longer benchmark, whereas applications with short panels may require a shorter one. The following command uses the prior 20 years as the rolling reference period:

```
. use rain.dta, clear
. wxsum rf_, type(rain) ini_month(05) fin_month(10) lr_years(20) ///
>     keep(hhid)
```

Changing `lr_years()` affects only the rolling deviations and *z*-scores. It does not affect the seasonal means, medians, variances, standard deviations, skewness measures, totals, counts, shares, dry-spell measures, GDD categories, or daily-temperature bin counts.

5.5 Long output

To produce long-shaped output, the user specifies `shape(long)`:

```
. use rain.dta, clear
. wxsum rf_, type(rain) ini_month(05) fin_month(10) ini_day(15) ///
>     fin_day(15) rain_threshold(1) keep(hhid) ///
>     shape(long)
```

The output has one row per `id-year`. The time variable is named `year`. Variables are named `mean`, `median`, and so on, rather than `mean_YYYY`, `median_YYYY`, etc.

6 Conclusions

`wxsum` reduces the amount of custom code needed to transform daily weather data into seasonal measures commonly used in socioeconomic analysis. It standardizes the construction of summary statistics, threshold-based measures, rainy-day counts, dry spells, GDD bins, daily temperature bins, and rolling weather-shock measures. The command handles cross-year seasons, missing data, and large datasets without requiring users to reshape or preprocess their data beyond the standard wide format. An implementation for R has also been developed, extending the same core functionality to users who work outside Stata or who integrate weather processing into R-based data pipelines.

While standardizing calculations, `wxsum` leaves substantive choices with the researcher by allowing users to customize the season definition, the thresholds for growing and killing degree days, GDD bin widths and endpoints, daily temperature bin counts and endpoints, the rainfall threshold, the length of the long-run window used to compute deviations and *z*-scores, and the shape of the output data frame. This division of labor is useful: the command reduces the likelihood of programming errors in repetitive weather-processing tasks, but it does not substitute for careful research design.

7 References

- Auffhammer, M., S. Hsiang, W. Schlenker, and A. Sobel. 2013. Using Weather Data and Climate Model Output in Economic Analyses of Climate Change. *Review of Environmental Economics and Policy* 7(2): 181–198.
- Benami, E., J. D. Michler, A. Josephson, M. Cecil, and G. Maskell. 2026. The Uses (and Misuses) of Earth Observation Data on Weather and Vegetation. In *Geospatial Impact Evaluation in Practice*, ed. A. BenYishay and K. Singh. Taylor & Francis.
- Bosilovich, M., R. Lucchesi, and M. Suarez. 2016. MERRA-2: File Specification. GMAO Office Note No. 9 (Version 1.1). http://gmao.gsfc.nasa.gov/pubs/office_notes.
- Chen, M., W. Shi, P. Xie, V. B. Silva, V. E. Kousky, R. W. Higgins, and J. E. Janowiak. 2008. Assessing objective techniques for gauge-based analyses of global daily precipitation. *Journal of Geophysical Research* 113: D04110.
- Dell, M., B. F. Jones, and B. A. Olken. 2014. What Do We Learn from the Weather? The New Climate-Economy Literature. *Journal of Economic Literature* 52(3): 740–798.
- Deschênes, O., and M. Greenstone. 2007. The Economic Impacts of Climate Change: Evidence from Agricultural Output and Random Fluctuations in Weather. *American Economic Review* 97(1): 354–85.
- Dinku, T. 2020. Remote sensing of African rainfall. *Oxford Research Encyclopedia of Climate Science* 30 July.
- Donaldson, D., and A. Storeygard. 2016. The View from Above: Applications of Satellite Data in Economics. *Journal of Economic Perspectives* 30(4): 171–98.
- Funk, C., P. Peterson, M. Landsfeld, D. Pedreros, J. Verdin, S. Shukla, G. Husak, J. Rowland, L. Harrison, A. Hoell, and J. Michaelsen. 2015. The climate hazards infrared precipitation with stations—a new environmental record for monitoring extremes. *Scientific Data* 2: 150066.
- Hersbach, H., B. Bell, P. Berrisford, G. Biavati, A. Horányi, J. Muñoz Sabater, J. Nicolas, C. Peubey, R. Radu, I. Rozum, A. Schepers, D. and Simmons, C. Soci, D. Dee, and J.-N. Thépaut. 2018. ERA5 hourly data on single levels from 1940 to present. Copernicus Climate Change Service (C3S) Climate Data Store (CDS). 10.24381/cds.adbb2d47.
- Josephson, A., J. D. Michler, T. Kilic, and S. Murray. 2026. The mismeasure of weather: Using earth observation data for estimation of socioeconomic outcomes. *Journal of Development Economics* 178: 103553.
- Mellon, J. 2025. Rain, Rain, Go Away: 194 Potential Exclusion-Restriction Violations for Studies Using Weather as an Instrumental Variable. *American Journal of Political Science* 69(3): 881–898.

- Michler, J. D., A. Josephson, E. Benami, P. Behrer, M. J. Cecil, S. Gourlay, R. Heilmayr, E. Kirchner, G. Maskell, and K. Singh. 2026. Treatment Geometry and Causal Identification with Earth Observation Data. In *Geospatial Impact Evaluation in Practice*, ed. A. BenYishay and K. Singh. Taylor & Francis.
- Michler, J. D., A. Josephson, T. Kilic, and S. Murray. 2021. Estimating the Impact of Weather on Agriculture. World Bank Policy Research Working Paper, No. 9867.
- . 2022. Privacy protection, measurement error, and the integration of remote sensing and socioeconomic survey data. *Journal of Development Economics* 158: 102927.
- Novella, N. S., and W. M. Thiaw. 2013. African Rainfall Climatology Version 2 for Famine Early Warning Systems. *Journal of Applied Meteorology and Climatology* 52(3): 588–606.
- Proctor, J., T. Carleton, and S. Sum. 2023. Parameter Recovery Using Remotely Sensed Variables. NBER Working Paper 30861.
- Sarsons, H. 2015. Rainfall and Conflict: A Cautionary Tale. *Journal of Development Economics* 115: 62–72.
- Schlenker, W., and M. J. Roberts. 2009. Nonlinear temperature effects indicate severe damages to U.S. crop yields under climate change. *Proceedings of the National Academy of Sciences* 106(37): 15594–15598.
- Tarnavsky, E., D. Grimes, R. Maidment, E. Black, R. P. Allan, M. Stringer, R. Chadwick, and F. Kayitakire. 2014. Extension of the TAMSAT Satellite-Based Rainfall Monitoring over Africa and from 1983 to Present. *Journal of Applied Meteorology and Climatology* 53(12): 2805–22.

Acknowledgments

We thank Brian McGreal for help in the early debugging of the `wxsum` command and Emanuele Clemente for final testing. Generative artificial intelligence tools were used during the final debugging and manuscript preparation process. Specifically, Anthropic’s Claude Opus 4.6 Thinking, OpenAI’s ChatGPT 5.5 Thinking, and Google Lab’s Jules based on Gemini 3.1 Pro were used in final debugging, testing, and validation. The same Claude and GPT models were used to edit and format the article’s text. The authors wrote, reviewed, and are responsible for the final content of the article and software.

About the authors

Jeffrey D. Michler is an Associate Professor in the Department of Agricultural and Applied Economics at the University of Arizona.

Anna Josephson is an Associate Professor in the Department of Agricultural and Applied Economics at the University of Arizona.

Aleksandr Michuda is a Visiting Assistant Professor at the Department of Economics at Swarthmore College.

Oscar Barriga-Cabanillas is an Economist at the World Bank.

Jeffrey C. Oliver is a Data Science Specialist in the University of Arizona Libraries.